Task 1: Guided Activity, Marks and grades toolkit

```
import Foundation

func average(of marks: Int...) -> Double {
   if marks.isEmpty {
      return 0
   }
   return Double(marks.reduce(0, +)) / Double(marks.count)
}


func grade(for average: Double) -> String {
   if average >= 90 {
      return "O"
   } else if average >= 80 {
      return "A+"
   } else if average >= 70 {
      return "A"
   } else if average >= 60 {
      return "B"
   } else {
      return "RA"
   }
}


func stats(of marks: [Int]) -> (min: Int, max: Int, average: Double)? {
   guard !marks.isEmpty,
       let minimum = marks.min(),
       let maximum = marks.max() else {
      return nil
   }


   let avg = Double(marks.reduce(0, +)) / Double(marks.count)
```

```
    return (minimum, maximum, avg)
}

func curve(_ marks: inout [Int], by bonus: Int = 5) {
    for i in marks.indices {
        marks[i] = min(marks[i] + bonus, 100)
    }
}

let avg = average(of: 85, 90, 78, 95)
print("Average:", avg)
print("Grade:", grade(for: avg))
print("Empty Average:", average(of:))

let marks = [75, 88, 96, 60]

if let result = stats(of: marks) {
    print("Minimum:", result.min)
    print("Maximum:", result.max)
    print("Average:", result.average)
}

var studentMarks = [75, 88, 96, 60]
curve(&studentMarks)
print("Marks after bonus:", studentMarks)
```

output:

Output

```
Average: 87.0
Grade: A+
Empty Average: (Function)
Minimum: 60
Maximum: 96
Average: 79.75
Marks after bonus: [80, 93, 100, 65]
```

Task 2: Practice Task, Text utilities:

```
import Foundation

func isPalindrome(_ text: String) -> Bool {
    let cleaned = text.lowercased().filter {
        $0.isLetter || $0.isNumber
    }
    return cleaned == String(cleaned.reversed())
}


func vowelCount(in text: String) -> Int {
    return text.lowercased().filter {
        "aeiou".contains($0)
    }.count
}


func wordCount(in text: String) -> Int {
    return text.split { $0.isWhitespace }.count
}


func wordCount(in text: String, minLength: Int) -> Int {
    return text.split { $0.isWhitespace }.filter { word in
        word.filter { $0.isLetter }.count >= minLength
    }.count
```

```
}

let sentence = "A man, a plan, a canal: Panama"

print("Palindrome:", isPalindrome(sentence))
print("Vowel Count:", vowelCount(in: sentence))
print("Word Count:", wordCount(in: sentence))
print("Words with at least 4 letters:",
    wordCount(in: sentence, minLength: 4))
```

Output

```
Palindrome: true
Vowel Count: 10
Word Count: 7
Words with at least 4 letters: 3
```

Task 3: Challenge Task, Functions as values

```
import Foundation
func makeMultiplier(_ factor: Int) -> (Int) -> Int {
  return { number in
    return number * factor
  }
}
let double = makeMultiplier(2)
let triple = makeMultiplier(3)

func apply(_ values: [Int], using transform: (Int) -> Int) -> [Int] {
  return values.map(transform)
}

let numbers = [1, 2, 3, 4, 5]
```

```
print("Original:", numbers)

print("Doubled:", apply(numbers, using: double))

print("Tripled:", apply(numbers, using: triple))


let squared = apply(numbers, using: { number in
    number * number
})


print("Squared:", squared)
```

Output

```
Original: [1, 2, 3, 4, 5]
Doubled: [2, 4, 6, 8, 10]
Tripled: [3, 6, 9, 12, 15]
Squared: [1, 4, 9, 16, 25]
```